

Also: Rayko Gueing doo Sauron

The text explains that software testing is not about proving that a program is free of errors, but about finding as many errors as possible. Testing every possible input and execution path is generally impossible because even simple programs can have an enormous number of combinations. Therefore, testing must consider both technical limitations and economic factors, such as time and available resources.

The author also emphasizes the psychology of testing. Testers believe their goal is to prove that a program works. They have an unconscious choice of test cases that are unlikely to fail. Instead, testers should assume that errors exist and actively try to expose them. A successful test is therefore one that finds an error that can be fixed, rather than one that simply produces the expected results.

The text then discusses the economics of testing. Since exhaustive testing is impossible, the goal should be to obtain the greatest number of useful results from a limited number of test cases. Two important approaches are introduced: black-box testing, which focuses on inputs, outputs, and specifications without considering the program's internal structure, and white-box testing, which examines the program's internal logic and execution paths. Finally, the text explains that neither exhaustive input testing nor exhaustive path testing can guarantee that a program is correct. There may be too many possible inputs and paths, and some errors depend on specific data or timing. Therefore, a practical testing strategy must combine different approaches and focus on finding the most important errors within realistic time and resource constraints.

Another important point is that a test should be considered successful when it provides useful information about the program, especially when it reveals an error. The text compares this idea to a medical examination: finding a problem allows it to be treated, while failing to identify the problem does not necessarily mean that the test was successful. This perspective encourages testers to actively challenge the program instead of simply confirming that it behaves as expected.